ONSITE PROJECT PRESENTATION

Fleet Simulation Engine

Vessels, docks and cargo, driven one JSON command at a time

Andy Chan

github.com/andychan1998/fleet-simulation-engine

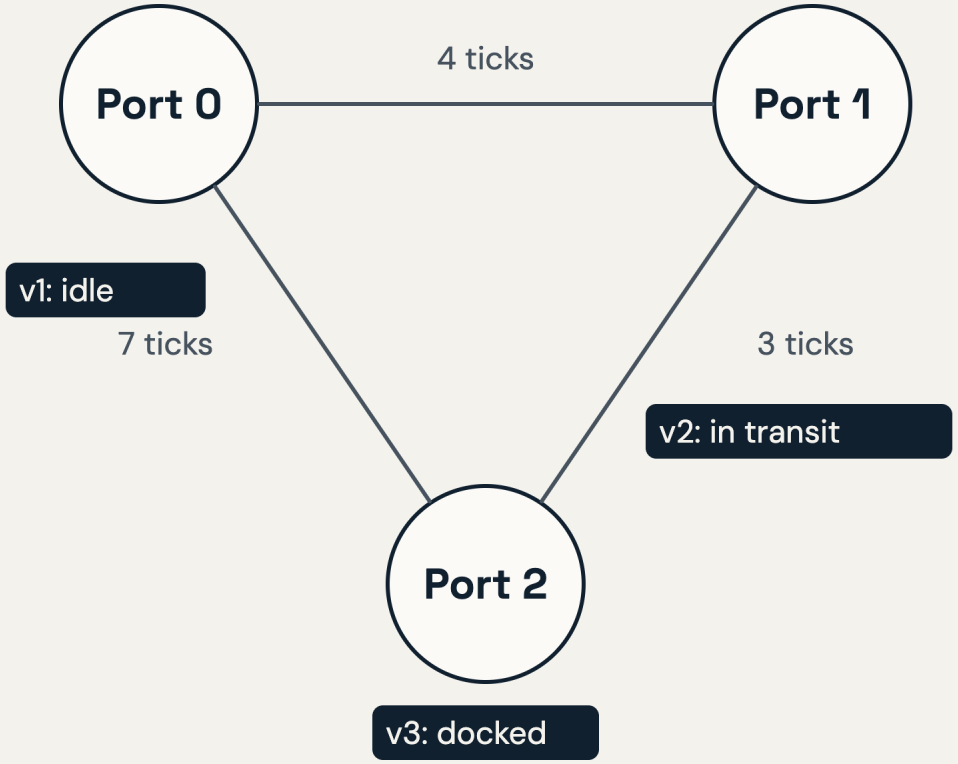
AGENDA

The next 25 minutes

1	**The basics** · a refresher, then the core model	5 min
2	**Design decisions** · where the spec left the choice to me	5 min
3	**Implementation** · modules, and how a command is handled	3 min
4	**Testing** · layers, coverage, mutation testing	5 min
5	**Performance** · speed, memory, what I'd improve	4 min
6	**Live demo** · then questions	3 min

THE BASICS

How the simulation works



Ports, with a table of travel times between every pair

Vessels, each idle at a port, in transit, or docked

Ticks: time moves one step per tick command, and only then

Commands: one JSON object per line in, one JSON line out, either the full state or an error

History: the state at any past tick can be read back

THE BASICS

Cargo and docks

1 · Cargo arrives

A manifest on init or tick puts units on a port's dock, each labeled only by its destination.

2 · Vessels move it

A docked vessel can load, unload or swap: one unit each way per command.

3 · Docks have a limit

Each dock handles a fixed number of operations per tick, shared by every vessel docked there.

4 · Delivery

Unloaded at its destination, cargo is delivered and leaves the simulation. Anywhere else, it waits on that dock.

THE BASICS

Two states: committed and planned

TICK T

Committed

The world as of the last tick. Every command is checked against it.

UNTIL THE NEXT TICK

Planned

Committed plus every command accepted since the last tick. This is what get_state returns.

WHEN TICK RUNS

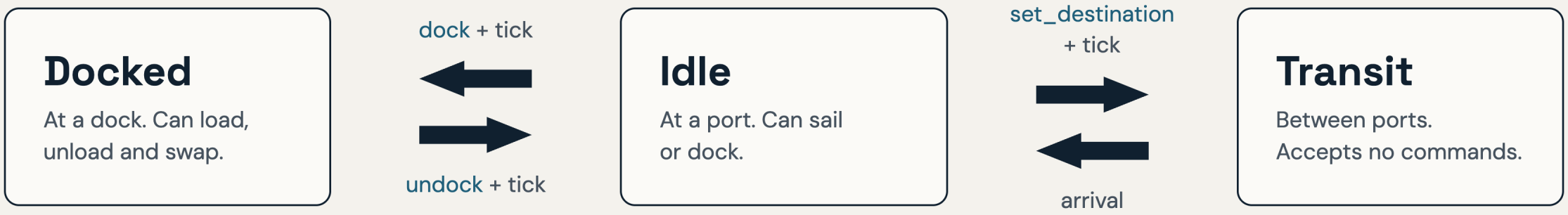
New committed

The plan becomes tick T+1's committed state and is saved to history.

History keeps every committed state, never changed afterwards, so get_state_at is a simple lookup.

THE BASICS

Three vessel states



Every change waits for the next tick. Until then it's pending, and a later command can replace or cancel it. That's where most of the open questions came from.

DESIGN DECISIONS · LEFT OPEN BY THE SPEC

Movement before the next tick

ID	Question	Decision	Rationale
D1	set_destination to the current port	Accepted: the vessel stays idle	A way to cancel a pending trip
D2	dock after dock	Accepted: nothing changes	Passes the spec's check; the vessel is already docked in the plan
D3	undock after dock	Accepted: cancels the pending dock	A way to undo a dock
D4	undock after undock	Accepted: nothing changes	Passes the spec's check; the vessel is already undocked in the plan
D5	dock after undock	Accepted: cancels the pending undock	A way to undo an undock

DESIGN DECISIONS · LEFT OPEN BY THE SPEC

Cargo and throughput

ID	Question	Decision	Rationale
D6	undock after load, unload or swap	Accepted: the cargo moves, then the vessel leaves	Commands take effect in order, so the cargo work came first
D7	load, unload or swap after undock	Rejected until a dock cancels the undock	In order, the vessel has already left in the plan
D8	load and unload the same cargo	Accepted: they cancel out, costing 0 operations	Throughput counts only the work the dock really does

DESIGN DECISIONS

Three more from building it

D9

A vessel loads cargo another vessel unloaded this tick

Accepted. It costs 2 operations, one per vessel.

Why: the plan updates with each command, so the cargo is already on the dock.

D10

A swap where only one half can succeed

Rejected. Nothing changes.

Why: a half-done swap isn't what was asked for.

D11

A swap with the same destination both ways

Always accepted. Nothing changes.

Why: it can't change anything, so a no-op is the simplest rule.

DESIGN DECISIONS · D8

Dock throughput counts net change

PER VESSEL, SINCE THE LAST TICK

`max(loaded, unloaded)`

A swap is one load plus one unload, so it costs 1.

Summed over the vessels docked there, it must stay within the dock's limit.

Commands this tick	Operations
Load 2 units	2
Load, then unload the same destination	0
Swap	1
A unloads, B loads the same unit	2

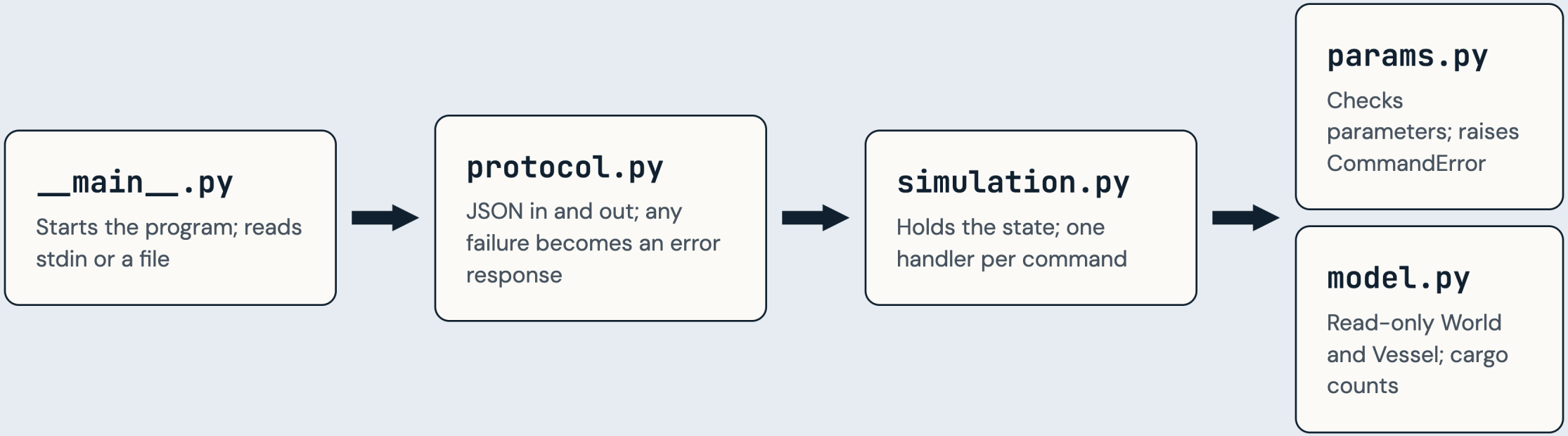
THE IDEA BEHIND MOST DECISIONS

Commands take effect in order.
A rejected command changes nothing.

Most of D1–D11 follow from one of these two rules.

IMPLEMENTATION

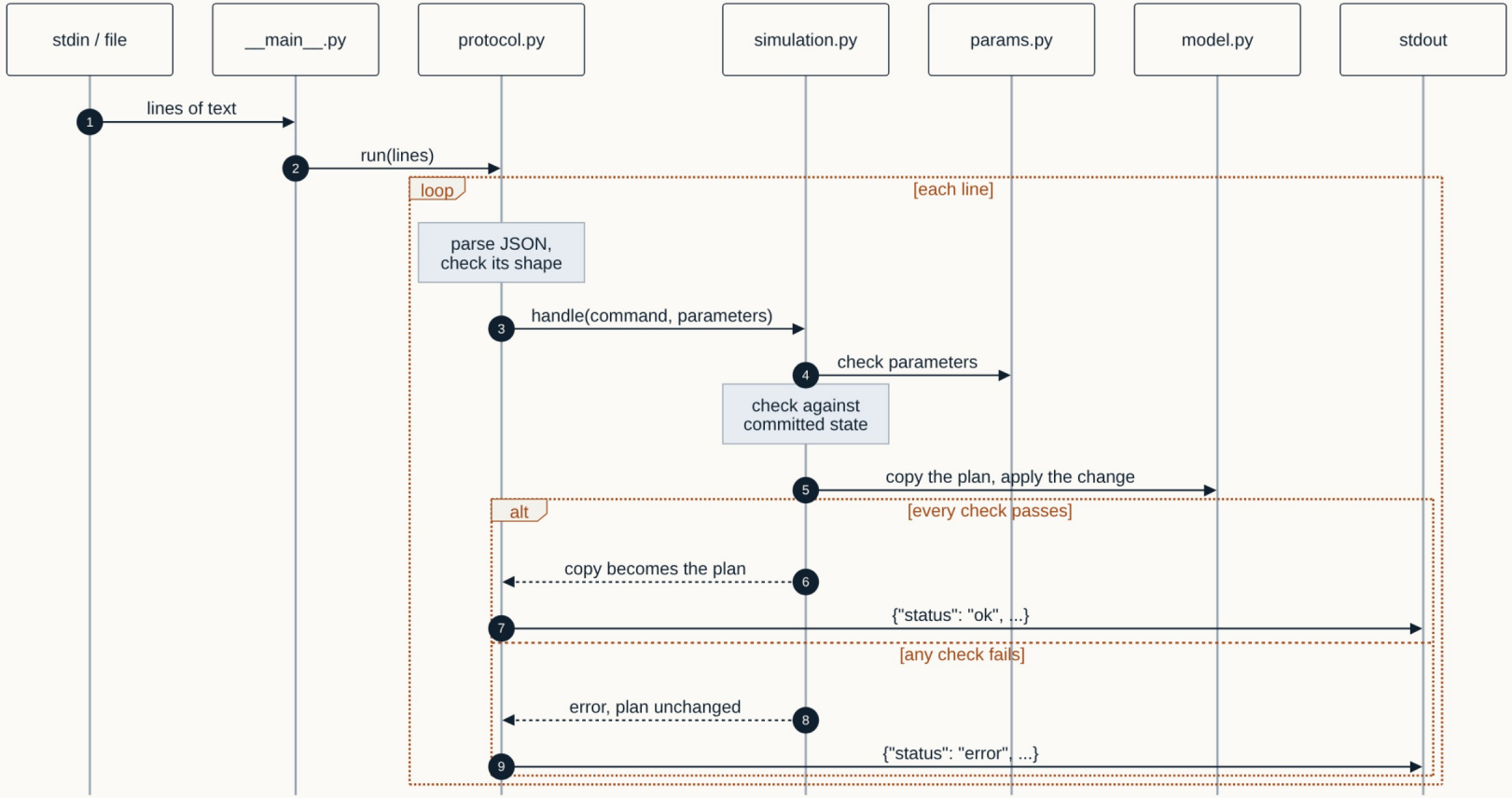
Five small modules



Python 3.9–3.14, standard library only: nothing to install.

IMPLEMENTATION

One command’s path through the code



IMPLEMENTATION

Every command is all or nothing

1

Parse the JSON and check the parameters

2

Check it against the committed state

3

Apply it to a copy of the planned state

4

Keep the copy only if every check passed

✓ **Accepted:** the copy becomes the planned state, and the response shows it.

⚠ **Rejected:** the copy is thrown away. Nothing changes; the response is an error message.

IMPLEMENTATION

States share what didn't change

Tick 5

v1	v2	v3	port 0	port 1
vessel	vessel	vessel	cargo table	cargo table

Tick 6, after v1 loads cargo at port 0

v1′	v2	v3	port 0′	port 1
new	shared	shared	**new**	shared

Read-only

Vessels, cargo tables and states can't be edited once built.

Copy only what changes

A command builds a new state with a new vessel and port table; the rest is shared.

All or nothing

If a check fails, the new state is discarded. Nothing was ever edited.

TESTING · ABOUT 3 SECONDS

Tests in layers

ID	Layer	What it checks
T1	Spec examples	Every example in the project document
T2	Design decisions	Each decision, D1–D11 and more
T3	Rules and edge cases	Shared dock limits, simultaneous arrivals, …
T4	Command matrix	56 cases: 8 vessel situations × 7 commands
T5	Random stress test	About 20,000 random commands, rules checked after each
T6	Command line	The real program: messy input, startup errors
T7–T9	Smaller checks	Output order, read-only history, safety net, tools

Every push: GitHub Actions runs them on Linux and macOS, Python 3.9–3.14

On request: T10 large world · T11 mutation test · T12 benchmark

TESTING

How I know the tests are thorough

100%

of lines and branches run by the tests

450 / 454

planted bugs caught; the other 4 can't change behavior

Mutation testing plants one small bug at a time, like a flipped comparison or a deleted line, and checks that some test fails.

PERFORMANCE

Cost grows in line with world size

World	Change	Read	Tick
5 ports, 10 vessels	0.04 ms	0.04 ms	0.05 ms
10 ports, 100 vessels	0.19 ms	0.18 ms	0.23 ms
50 ports, 1,000 vessels	1.7 ms	1.6 ms	2.1 ms
100 ports, 5,000 vessels	7.9 ms	7.5 ms	8.6 ms

O(n) per command, where n is vessels plus cargo entries: under 2 µs per vessel.

A change costs about a read. Building the full-state response, which the spec requires, is the main cost.

Median per command, engine only (tools/benchmark.py). MacBook Pro, Apple M1 Pro, Linux VM, Python 3.10.

PERFORMANCE

History stays small and safe

60 MB

of history per 1,000 ticks, at 100 ports and 1,000 vessels

Each tick stores only the vessels and ports that changed, plus a small index.

Space is O(t × n), but each tick holds references, not copies.

States are read-only, so history can't be changed by accident, and get_state_at is a simple lookup.

Measured with tools/benchmark.py over 1,000 ticks. MacBook Pro, Apple M1 Pro, Linux VM, Python 3.10.

NEXT STEPS

What I’d improve next

Faster responses

Every response lists all vessels and port cargo, but most haven’t changed since the last one. Each part could be built once and reused.

Only what changed would need building again.

Less memory

Store only the differences each tick, plus an occasional full snapshot.

Trade-off: get_state_at would have to rebuild past states.

More realism

- Cargo capacity per vessel
- Speed per vessel
- Berth limits per dock
- Cargo with deadlines

LIVE DEMO

Watching it run

```
python tools/view.py --step examples/demo.jsonl
```

What to watch for

- Each command, then the planned state it produces
- Each tick, as the plan becomes committed
- Two deliberate rejections: a vessel that isn’t idle, and a transfer over a dock’s limit
- Earlier ticks read back from history

WRAP-UP

How I worked

My part

- The design decisions, D1–D11
- Reviewing all of Claude's work
- The design document

Claude's part

- The code, tests and tools
- The README and these slides
- Running the measurements

How I checked it

- Tests in layers, run on every push
- Mutation testing: do the tests catch real bugs?
- Speed and memory, measured

THANK YOU

Questions?

Then let’s work on the code live.

Andy Chan

github.com/andychan1998/fleet-simulation-engine